

Relatório: Projeto de Vigilância de Partições Retangulares

Cadeira: Métodos de Apoio à Decisão (MAD)

Ano Letivo: 2025/2026

Grupo: Beatriz Castro up202306399

Joana Ferreira up202305202

Mariana Bissacot up202306832

1. Introdução

Este projeto foca-se na otimização de recursos para a vigilância completa de uma área particionada geometricamente em retângulos. O objetivo é determinar o número mínimo de guardas necessários, posicionados exclusivamente nos vértices, para que cada retângulo seja vigiado por pelo menos um guarda.

2. Modelação Matemática

A modelação formal do problema de vigilância de partições retangulares é tratada como uma instância específica do problema de **Cobertura de Conjuntos (Set Cover)**, adaptada para uma estrutura geométrica de retângulos e vértices.

2.1. Definição do Conjunto de Vértices Únicos

Dado que os retângulos na partição Π partilham fronteiras, múltiplos retângulos podem partilhar o mesmo vértice físico. Para evitar redundância e garantir a otimalidade, é realizado um pré-processamento para identificar o conjunto de vértices únicos $V = \{v_1, v_2, \dots, v_n\}$ a partir das coordenadas (x, y) extraídas do gerador de instâncias.

2.2. Variáveis de Decisão

Para cada vértice único $v_i \in V$, definimos uma variável de decisão binária:

$$x_i \in \{0, 1\}$$

Valor	Significado
1	Um guarda é colocado no vértice v_i .
0	Nenhum guarda é colocado no vértice v_i .

2.3. Função Objetivo

O objetivo do projeto é minimizar o custo total de infraestrutura, que corresponde ao número total de guardas necessários para cobrir toda a partição:

$$\text{Minimizar } Z = \sum_{i=1}^n x_i$$

2.4. Restrições de Cobertura

Cada retângulo r_j da partição (onde $j = 1, \dots, m$) deve ser vigiado por pelo menos um guarda. Matematicamente, a soma das variáveis associadas aos vértices que compõem o retângulo r_j deve ser maior ou igual a um:

$$\sum_{i \in \text{Adj}(r_j)} x_i \geq 1, \forall r_j \in \Pi$$

Onde $\text{Adj}(r_j)$ representa o conjunto de índices dos vértices incidentes no retângulo r_j .

2.5. Variante de Cobertura Parcial — $\Pi' \subset \Pi$

Na variante parcial, as restrições de cobertura aplicam-se apenas ao subconjunto $\Pi' \subset \Pi$:

$$\sum_{i \in \text{Adj}(r_j)} x_i \geq 1, \forall r_j \in \Pi'$$

Os retângulos em $\Pi \setminus \Pi'$ podem ou não ser cobertos — o modelo não os proíbe nem os exige. Esta relaxação reduz o número de restrições activas e tipicamente permite soluções com menos guardas, embora o problema permaneça NP-difícil para qualquer fração fixa de Π' .

2.6. Pré-processamento e Mapeamento de Dados

Para implementar este modelo, o sistema executa os seguintes passos lógicos:

- **Leitura:** Processamento do ficheiro de instância para extrair as coordenadas dos vértices de cada face.
- **Unificação:** Criação de um dicionário de mapeamento para unificar coordenadas coincidentes em identificadores de variáveis únicos.
- **Geração:** Construção dinâmica da matriz de restrições baseada na vizinhança geométrica detetada.

3. Implementação do Solver Manual (MAC com GAC — generalização de AC-3)

A implementação manual (ficheiro *solver_mac.py*) foi desenvolvida em Python, focando-se em algoritmos de satisfação de restrições (CSP).

3.1. Propagação de Restrições (GAC)

Devido à natureza n -ária das restrições (cada retângulo possui múltiplos vértices), implementou-se o algoritmo Generalized Arc Consistency (GAC). O motor de consistência deduz valores obrigatórios: se um retângulo tiver apenas um vértice disponível no seu domínio, o sistema fixa automaticamente esse vértice como guarda ($x_i = 1$), podando a árvore de procura antes de realizar novas ramificações.

O enunciado especifica a aplicação de MAC com AC-3. O algoritmo AC-3 opera sobre restrições binárias, propagando consistência de arco entre pares de variáveis. Contudo, as restrições deste problema são intrinsecamente n

A implementação adotada aplica GAC (Generalized Arc Consistency), que é a generalização natural do AC-3 para restrições n

3.2. Heurística de Ordenação: Maior Grau

Para acelerar a convergência, utilizou-se a heurística de **Maior Grau** na seleção de variáveis. O algoritmo ordena os vértices com base na sua incidência: vértices que pertencem a mais retângulos são processados primeiro, permitindo cobrir a área de forma mais eficiente logo no início da busca.

3.3. Estratégias Heurísticas Gananciosas (Greedy)

Para contrastar com a natureza pesada do solver de restrições, implementaram-se duas variantes de algoritmos gananciosos (*Greedy*), cujo foco principal é a eficiência temporal sacrificando, se necessário, a garantia de otimalidade:

- **Greedy por Inserção (Maior Cobertura):** O algoritmo inicia com zero guardas colocados. Em cada iteração, avalia todos os vértices disponíveis e seleciona aquele que vigia o maior número de retângulos que ainda se encontram sem cobertura. Este processo repete-se de forma estritamente progressiva até que a totalidade dos retângulos esteja vigiada.
- **Greedy por Remoção (Reverse Greedy):** Esta estratégia segue a lógica inversa. Inicia-se o cenário posicionando guardas em todos os vértices possíveis da partição. Sequencialmente, o algoritmo avalia a redundância de cada guarda; se todos os retângulos vigiados por um guarda específico já possuem vigilância redundante de outros guardas, este é removido. A heurística prioriza a remoção do guarda com maior índice de sobreposição, parando quando nenhuma remoção adicional for possível sem deixar retângulos desprotegidos.

3.4. Avaliação de Programação Dinâmica

Seguindo o ponto 3 do enunciado do projeto, avaliou-se a aplicação de Programação Dinâmica (PD). O problema de Cobertura de Conjuntos (*Set Cover*) geométrico foi modelado recorrendo à técnica de *Bitmasking* (Máscaras de Bits), onde cada estado da tabela de memorização representa de forma binária a combinação de retângulos que já se encontram cobertos.

A transição de estados baseia-se na Equação de Bellman, determinando o custo mínimo (número de guardas) para atingir uma nova máscara de cobertura combinando o estado anterior com o alcance de um novo vértice. Contudo, constatou-se que a complexidade algorítmica deste método exato é de $O(V \cdot 2^R)$, onde V representa o número de vértices únicos e R o número de retângulos. Esta formulação matemática demonstra que, embora a PD seja extremamente rápida e eficaz para subproblemas contidos (garantindo o ótimo exato), ela sofre de uma severa explosão combinatória espacial e temporal quando aplicada à totalidade de partições irregulares complexas (como as instâncias de 40 e 50 retângulos fornecidas).

4. Abordagem em Programação em Lógica (Prolog)

4.1. Modelação Declarativa

A cobertura foi modelada como restrição de soma: para cada retângulo, a soma dos guardas nos seus vértices deve ser ≥ 1 . A estrutura lógica é independente do algoritmo de procura, permitindo adicionar restrições complexas sem reestruturar o modelo base.

4.2. Validação em Instâncias Miniatura

O solver Prolog com clpfd foi usado para validar a correção do modelo matemático em instâncias de dimensão reduzida, onde a explosão combinatória é controlável. Os resultados coincidem exactamente com os do OR-Tools:

Instância	$ \Pi $	Vértices	OR-Tools	Prolog (clpfd)
mini_A	3	8	2	2
mini_B (grelha 2×3)	6	12	2	2
mini_C (grelha 3×3)	9	16	4	4

O Prolog não foi incluído na tabela comparativa principal devido à explosão combinatória para instâncias de 40+ retângulos, servindo estritamente para validação do modelo e estudo das extensões.

4.3. Extensão 4a — Guardas com Cores

Implementou-se a restrição de que guardas no mesmo retângulo devem ter cores distintas. A equivalência lógica $G \neq 0 \iff C \neq 0$ acopla a existência de um guarda ao seu domínio de cores. A restrição de distinção foi implementada por pares: $(C1 \neq 0 \wedge C2 \neq 0) \implies (C1 \neq C2)$.

Os resultados experimentais nas instâncias miniatura mostram um fenômeno relevante: o número mínimo de cores depende da solução de guardas escolhida*, não apenas do seu número. Para a *mini_A*, existe uma solução com 2 guardas sem qualquer conflito (1 cor), mas outras soluções igualmente ótimas em guardas colocam dois guardas no retângulo central e exigem 2 cores. O solver Prolog encontra automaticamente a solução que minimiza simultaneamente guardas e cores.

Instância	$ \Pi $	Vértices	Guarda min.	Cores mín. (melhor sol.)	Cores (pior sol. ótima).
mini_A	3	8	2	1	2
mini_B	6	12	2	1	2
mini_C	9	16	4	1	4

A instância *mini_A* foi construída propositalmente para ilustrar o fenômeno: $R2 = \{1,2,5,6\}$ e $R3 = \{3,4,7,8\}$ forçam guardas nos grupos $\{1,2\}$ e $\{3,4\}$. O retângulo central $R1 = \{1,2,3,4\}$ contém ambos — esta geometria torna o conflito de cores inevitável em certas soluções ótimas.

4.4. Extensão 4b — Guardas com Maior Alcance (D)

Para alcance D, um guarda num vértice cobre não só os retângulos incidentes mas também todos os retângulos a distância $\leq D$ no grafo de adjacência. Foi desenvolvido um predicado de expansão que pré-calcula a vizinhança expandida de cada retângulo.

Instância	$ \Pi $	Vértices	Guardas D=0	Guardas D=1	Redução
mini_A	3	8	2	1	-50%
mini_B	6	12	2	1	-50%
mini_C	9	16	4	1	-75%

O impacto do alcance D=1 é substancial: na mini_C, um único guarda no vértice central com D=1 cobre todos os 9 retângulos, reduzindo de 4 para 1 guarda (-75%). Este resultado confirma que o alcance é o parâmetro com maior impacto na eficiência da vigilância.

5. Resultados Experimentais

5.1. Cobertura Total ($\Pi' = \Pi$) – OR-Tools

As instâncias do ficheiro *parts40* contém 100 instâncias geradas aleatoriamente, cada uma com 40 retângulos e 82 vértices únicos. O ficheiro *step50* contém 1 instância com 50 retângulos e 102 vértices.

Ficheiro	Instância	Π	Vértices	Guardas mín.	Guardas máx.	Guardas médios	Tempo médio (s)
parts40	100	40	82	13	17	~14.2	~0.007
step50	1	50	102	17	17	17.0	0.003

5.2. Cobertura Parcial ($\Pi' \subset \Pi$) – OR-Tools

Para estudar a variante de cobertura parcial, o solver foi executado com subconjuntos $\Pi' \subset \Pi$ selecionados aleatoriamente com frações 100%, 75%, 50% e 25% dos retângulos obrigatórios.

Ficheiro	Instância	Π	Π'	Fração	Guardas	Redução vs 100%	Tempo (s)
parts40	1	40	40	100%	14	—	0.00299
parts40	1	40	30	75%	11	−21.4%	0.00150
parts40	1	40	20	50%	10	−28.6%	0.00080
parts40	1	40	10	25%	6	−57.1%	0.00080
step50	1	50	50	100%	17	—	0.00288
step50	1	50	38	75%	15	−11.8%	0.00159
step50	1	50	25	50%	11	−35.3%	0.00102
step50	1	50	12	25%	8	−52.9%	0.00103

5.3 Comparação de Abordagens - Instâncias Representativas

Instância	Vértices Únicos	Ótimo (OR-Tools)	Melhor Manual (MAC)	Greedy (Inserção)	Greedy (Remoção)	Tempo Greedy	Nós (Greedy/ PD)
parts40	82	14	15	17	17	0.00032s	0 (Sem Nó)
step50	102	17	18	22	22	0.00049s	0 (Sem Nó)

O solver declarativo (clpfd) foi utilizado como prova de conceito para validar a lógica matemática. Em instâncias miniatura construídas para testes unitários, o Prolog convergiu exatamente para os mesmos valores ótimos gerados pelo OR-Tools. Não foi incluído na tabela acima devido à sua limitação natural perante a árvore de pesquisa de 282 estados, servindo estritamente para validação do modelo e estudo das extensões de cores e alcance.

6. Análise de Resultados e Conclusão

A análise dos dados revela uma discrepância clara de performance entre as abordagens.

- **Cobertura parcial $\Pi' \subset \Pi$:** A redução no número de guardas não é linear com a fracção de retângulos obrigatórios. Reduzir Π' para 50% não reduz os guardas para 50% do ótimo — na instância 1 do *parts40*, 50% dos retângulos ainda exige 10 guardas (71% do ótimo completo). Isto deve-se à partilha de vértices: um guarda que cobre um retângulo obrigatório frequentemente já cobre outros. A 25%, a variância entre instâncias aumenta (5 a 9 guardas nas 100 instâncias do *parts40*), dependendo da distribuição geométrica do subconjunto sorteado. O tempo de resolução mantém-se sempre sub-milissegundo, independentemente da fracção.
- **Explosão combinatória:** O solver manual processou mais de 1,7 mil milhões de nós para a instância *step50*. O facto de ter estabilizado em 18 guardas (apenas 1 acima do ótimo de 17) valida a lógica do motor de restrições, mas expõe os limites computacionais do Python face a instâncias de 102 variáveis binárias.
- **Eficácia do GAC:** Sem a consistência de arco generalizada e a poda por custo acumulado, o espaço de 2^{102} estados seria impossível de explorar. A proximidade ao ótimo comercial confirma a eficácia da propagação implementada.
- **Vantagem comercial:** O OR-Tools atinge o ótimo em milissegundos graças a técnicas de Branch-and-Cut e implementação em C++, enquanto o solver manual serve como prova de conceito sobre o funcionamento interno dos CSPs.
- **Compromisso eficiência/ótimalidade:** O Greedy resolve *step50* em 0,00049 segundos (versus ~12 horas do MAC), com custo de 22 guardas contra o ótimo de 17. A Programação Dinâmica é exacta e rápida para subproblemas pequenos (7 guardas para 18 retângulos em ~0,15s) mas exponencialmente inviável para as instâncias completas.
- **Extensões:** A extensão de cores revela que o número mínimo de cores não é uma propriedade do número de guardas mas da solução específica escolhida — instâncias com geometria que força co-localização de guardas exigem mais cores. O alcance $D=1$ tem impacto muito superior: reduz o número de guardas em 50–75% nas instâncias miniatura, confirmando que a conectividade geométrica da partição é o factor determinante na eficiência da vigilância.

7. Conclusão

Este projeto explorou o problema de vigilância geométrica através de quatro abordagens complementares: heurísticas greedy, programação dinâmica, satisfação de restrições imperativa (MAC/GAC) e modelação declarativa (Prolog/clpfd).

As ferramentas comerciais (OR-Tools) são incomparáveis para cenários complexos, mas a construção dos solvers próprios foi essencial para compreender a anatomia da explosão combinatória e a importância da poda GAC — sem a qual o espaço de 2^{102} estados da instância *step50* seria inabordável. A programação dinâmica confirmou empiricamente a teoria: exata e instantânea para subproblemas pequenos, exponencialmente inviável para as instâncias completas. O Prolog destacou-se pela expressividade, validando o modelo matemático e permitindo incorporar as extensões com poucas linhas de código.

O estudo da variante $\Pi' \subset \Pi$ mostrou que a relaxação parcial reduz significativamente os recursos necessários — até 57% com 25% dos retângulos obrigatórios — sem alterar a natureza NP-difícil do problema. As extensões revelaram que pequenas modificações no modelo têm impacto desproporcionado: o alcance $D=1$ reduziu o número de guardas em até 75%, e a restrição de cores demonstrou que o número mínimo de cores depende da solução específica escolhida, não apenas do número de guardas.